

Тезисы доклада для предзащиты

Тема: Разработка CLI и веб-интерфейса для системы нагрузочного тестирования

Автор: Адаменко Семён Сергеевич

Научный руководитель: Государев Илья Борисович, к.п.н., доцент

1. Вступление

Уважаемые члены комиссии, тема моей выпускной квалификационной работы — «Разработка CLI и веб-интерфейса для системы нагрузочного тестирования».

Работа посвящена разработке инструмента Tsunami — программного средства для нагрузочного тестирования HTTP и gRPC сервисов, распространяемого в виде единого бинарного файла без внешних зависимостей и оснащённого встроенным веб-интерфейсом с визуализацией метрик в реальном времени.

2. Актуальность темы

Актуальность работы обусловлена тем, что нагрузочное тестирование является обязательным этапом обеспечения качества современных веб-сервисов. Оно позволяет выявить узкие места системы до выхода в промышленную эксплуатацию, оценить поведение при пиковых нагрузках и проверить соответствие нефункциональным требованиям.

Существующие инструменты имеют существенные ограничения: Apache JMeter и Gatling требуют установки среды выполнения Java, k6 — JavaScript-рантайма, а инструменты Vegeta и hey поддерживают только протокол HTTP и лишены веб-интерфейса для визуального мониторинга теста. Таким образом, на рынке отсутствует инструмент, одновременно обеспечивающий поставку в виде единого бинарного файла без зависимостей, поддержку HTTP и gRPC, встроенный веб-интерфейс с потоковой передачей метрик и интеграцию в CI/CD-пайплайны через REST API.

3. Проблема существующего подхода

Среди рассмотренных решений инструменты с поддержкой gRPC (JMeter, Gatling) требуют установки JVM, что усложняет развёртывание и исключает автономное использование в контейнерных средах. Лёгкие инструменты (Vegeta, hey) работают только с HTTP и не предоставляют никакого пользовательского интерфейса для наблюдения за тестом в процессе его выполнения. k6 поддерживает REST API, но зависит от JavaScript-рантайма и не имеет встроенного веб-интерфейса.

Таким образом, проблема заключается не в отсутствии инструментов нагрузочного тестирования как таковых, а в том, что ни один из них не сочетает минимальные зависимости, полную поддержку поддерживаемых протоколов, наглядный веб-интерфейс и пригодность для CI/CD одновременно.

4. Цель и задачи работы

Целью работы является разработка инструмента нагрузочного тестирования Tsunami, распространяемого в виде единого бинарного файла без внешних зависимостей и обеспечивающего поддержку HTTP и gRPC, встроенный веб-интерфейс с визуализацией метрик в реальном времени и интеграцию в CI/CD-пайплайны через REST API.

Для достижения цели были поставлены следующие задачи:

1. Провести сравнительный анализ существующих инструментов нагрузочного тестирования.
 2. Сформулировать функциональные и нефункциональные требования к разрабатываемому инструменту.
 3. Спроектировать архитектуру системы: движок нагрузочного тестирования, CLI-слой и серверную часть.
 4. Реализовать модуль нагрузочного тестирования с поддержкой протоколов HTTP и gRPC.
 5. Реализовать CLI-интерфейс и веб-интерфейс системы для управления нагрузочными тестами и мониторинга метрик в реальном времени.
 6. Провести тестирование разработанного инструмента и оценить его производительность.
-

5. Теоретическая основа

В теоретической части были рассмотрены основные механизмы, необходимые для разработки инструмента нагрузочного тестирования.

К ним относятся: модель параллелизма Go и статическая компиляция без зависимостей среды выполнения; паттерн «пул воркеров» для параллельной отправки запросов; алгоритм «дырявого ведра» (leaky bucket) для равномерного ограничения скорости; протокол WebSocket для потоковой передачи метрик; механизм репликации gRPC; а также подходы к вычислению перцентилей задержки на основе кольцевого буфера.

Ключевой вывод теоретической части: для обеспечения точного ограничения скорости при высоких нагрузках алгоритм leaky bucket предпочтительнее простого таймера `time.Ticker`, поскольку компенсирует накопленное отставание и обеспечивает равномерный поток запросов.

6. Идея разработанного решения

В рамках работы был разработан инструмент Tsunami с трёхуровневой архитектурой.

Основная идея решения заключается в том, чтобы объединить в одном бинарном файле движок нагрузочного тестирования, CLI-интерфейс и HTTP-сервер с встроенным React-приложением. Веб-интерфейс встраивается в исполняемый файл через директиву `go:embed`, что исключает необходимость во внешних статических активах и упрощает развёртывание до запуска одной команды.

Такой подход позволяет использовать инструмент как в автономном режиме через терминал, так и в браузерном режиме с визуальным мониторингом, не меняя способ установки.

7. Архитектура решения

Архитектура системы включает три слабосвязанных компонента.

CLI-уровень реализован с помощью фреймворка Cobra и включает три подкоманды: `tsunami attack` для запуска HTTP-теста из терминала, `tsunami grpc` для тестирования gRPC-сервисов и `tsunami serve` для запуска HTTP-сервера с веб-интерфейсом. CLI и HTTP-сервер используют один и тот же движок и не знают друг о друге.

Движок нагрузочного тестирования (`internal/attack`) реализован по паттерну «пул воркеров»: N горутин параллельно потребляют задания из канала, скорость регулируется через `uber/ratelimit`. Метрики накапливаются в структуре `GlobalMetrics` с кольцевым буфером на 100 000 замеров.

HTTP-сервер предоставляет 8 REST-эндпоинтов для управления тестами и WebSocket-хаб с интервалом трансляции 50 мс. React SPA встраивается в бинарь через `go:embed`.

8. Основные преимущества

Разработанный инструмент обеспечивает несколько практических преимуществ.

Первое — единый бинарный файл без зависимостей: инструмент компилируется с `CGO_ENABLED=0` и не требует установки JVM, Python, Node.js или иных рантаймов.

Второе — поддержка HTTP и gRPC в одном дистрибутиве, что устраняет необходимость держать два отдельных инструмента.

Третье — встроенный веб-интерфейс с реальным временем обновления: метрики передаются через WebSocket с задержкой 50 мс и отображаются в браузере без перезагрузки страницы.

Четвёртое — интеграция в CI/CD: REST API позволяет управлять тестами программно, без необходимости интерактивного ввода.

Пятое — кросс-платформенная дистрибуция для 6 платформ (Linux, macOS, Windows × amd64, arm64) через GitHub Releases и Homebrew.

9. Тестирование

В практической части была проведена проверка корректности и производительности разработанного решения.

Тестирование включало два уровня: юнит-тесты функции вычисления перцентилей `CalculateAllPercentiles` — 13 тест-кейсов, покрывающих граничные случаи (пустой срез, один элемент, равные значения, случайные данные); и интеграционные тесты с использованием библиотеки `testcontainers-go`, которые запускают реальный Docker-контейнер с сервисом `httpbin` и верифицируют точность классификации ошибок, корректность счётчиков метрик и соответствие REST API заявленным контрактам.

Все тесты выполняются в среде GitHub Actions CI при каждом коммите и перед каждым релизом. Кроме того, была проведена оценка производительности: при 50 параллельных воркерах инструмент стабильно обеспечивает более 1 000 запросов в секунду.

10. Результаты работы

В результате выполнения работы разработан инструмент нагрузочного тестирования Tsunami, который позволяет тестировать HTTP и gRPC сервисы через единый бинарный файл без внешних зависимостей.

Были решены следующие задачи: проведён сравнительный анализ пяти инструментов; сформулированы функциональные и нефункциональные требования; спроектирована трёхуровневая архитектура и разработаны UML-диаграммы; реализован движок нагрузочного тестирования с поддержкой HTTP и gRPC; реализованы CLI-интерфейс и веб-интерфейс; настроены CI/CD и автоматическая публикация для 6 платформ.

Достигнутые показатели: производительность более 1 000 RPS при 50 воркерах; размер бинаря около 12 МБ, включая встроенный React SPA; задержка WebSocket-трансляции 50 мс.

11. Заключение

Таким образом, в работе была решена задача разработки лёгкого инструмента нагрузочного тестирования, устраняющего ограничения существующих решений.

Разработанный инструмент не претендует на замену специализированных платформ уровня JMeter для сложных сценарных тестов. Его назначение — обеспечить быстрый старт нагрузочного тестирования HTTP и gRPC сервисов без зависимостей среды выполнения и с немедленной визуализацией результатов в браузере.

Главный результат работы состоит в том, что нагрузочное тестирование становится доступным в любой среде: локальной, контейнерной или CI/CD — за счёт единого самодостаточного бинарного файла и встроенного веб-интерфейса.

Спасибо за внимание. Готов ответить на вопросы.